

AI in signal and audio processing

Thomas de Jaeger

thomas.de-jaeger@epita.fr

Syllabus

- **Session 1:** Introduction & Fundamentals of Signal/Audio
- **Session 2:** Audio Features & Librosa
- **Session 3:** Audio Processing with Torchaudio
- **Session 4:** Sound Detection and Classification
- **Session 5:** Sequence Modeling with RNNs and LSTMs
- **Session 6:** Automatic Speech Recognition (ASR)
- **Session 7:** Speaker and Emotion Recognition
- **Session 8:** Project presentation

Sound Event Detection & Classification

Session 4

Sound Event Detection & Classification

Session 4

Objectives:

- Understand the core challenges of Sound Event Detection (SED)
- Learn tools, datasets, and feature extraction methods
- Discover architectures like CNNs for audio classification
- Practice model training, evaluation, and improvement techniques

Applications

- Smart assistants (e.g., Alexa, Google Home)
- Urban surveillance (e.g., gunshot detection)
- Wildlife monitoring (animal calls)
- Health (e.g., cough or apnea detection)
- Sound-based content tagging in videos

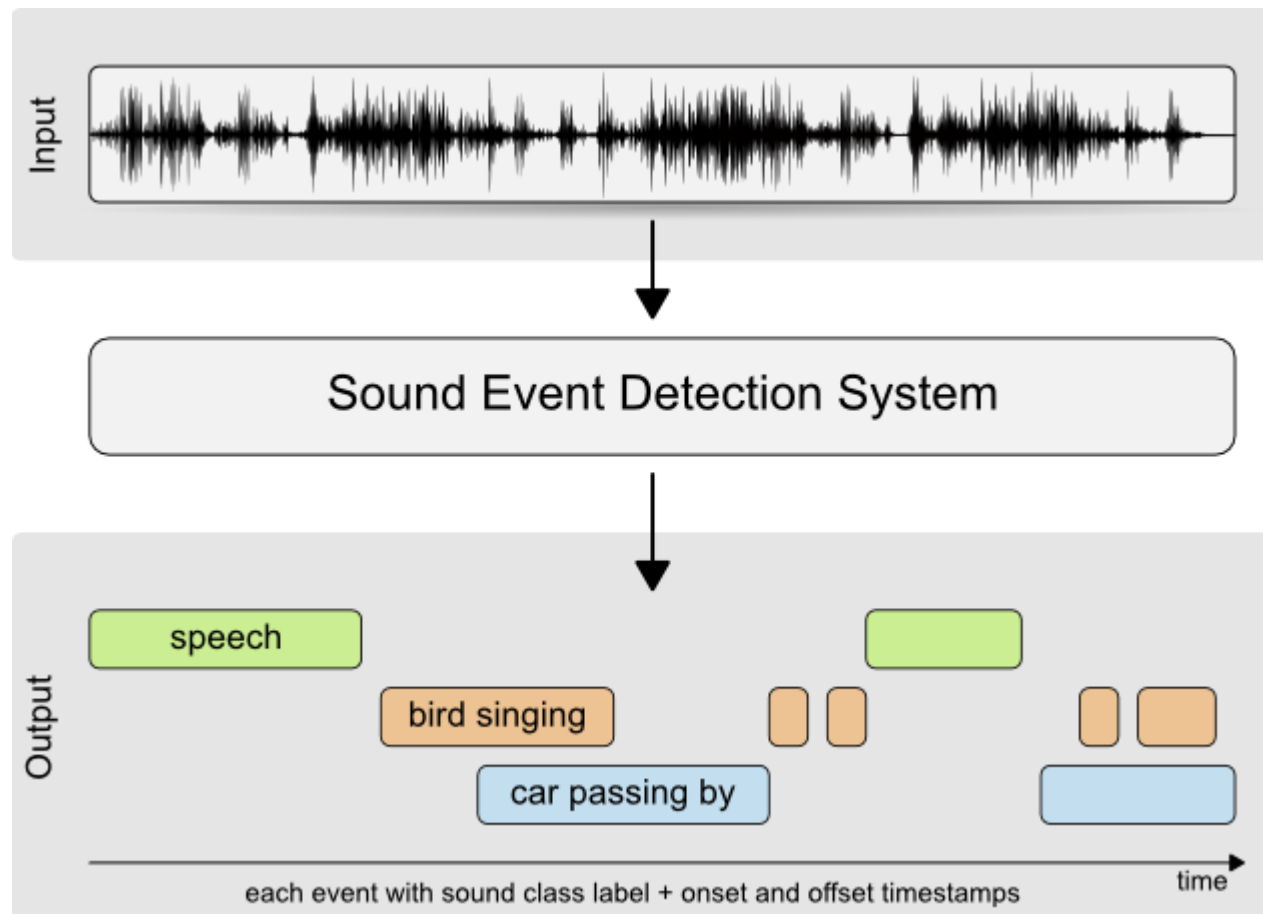
What is Sound Event Detection (SED)?

SED aims to **identify and locate sound events** in an audio stream

May be:

- **Classification:** What sound is present?
- **Detection:** When does it happen?
- **Multi-label:** More than one event at a time

It's the **audio equivalent** of object detection in images.



Tools and libraries

- torchaudio, librosa — feature extraction
- torch, tensorflow — deep learning
- pyannote-audio, openl3, panns — pretrained SED models

CNNs for Classification

Convolutional Neural Networks (CNNs) are powerful architectures designed specifically for **image and spatial data**. Unlike standard neural networks that treat all inputs independently, CNNs process images by scanning small regions using **convolutional filters**. This allows them to **capture local patterns** like edges, shapes, and textures effectively. **For audio, features are treated as images!!**

A typical CNN is composed of the following layers:

- **Input Layer:**

Receives the features/image.

- **Convolutional Layer:**

Applies filters to extract features.

- **Activation Function:**

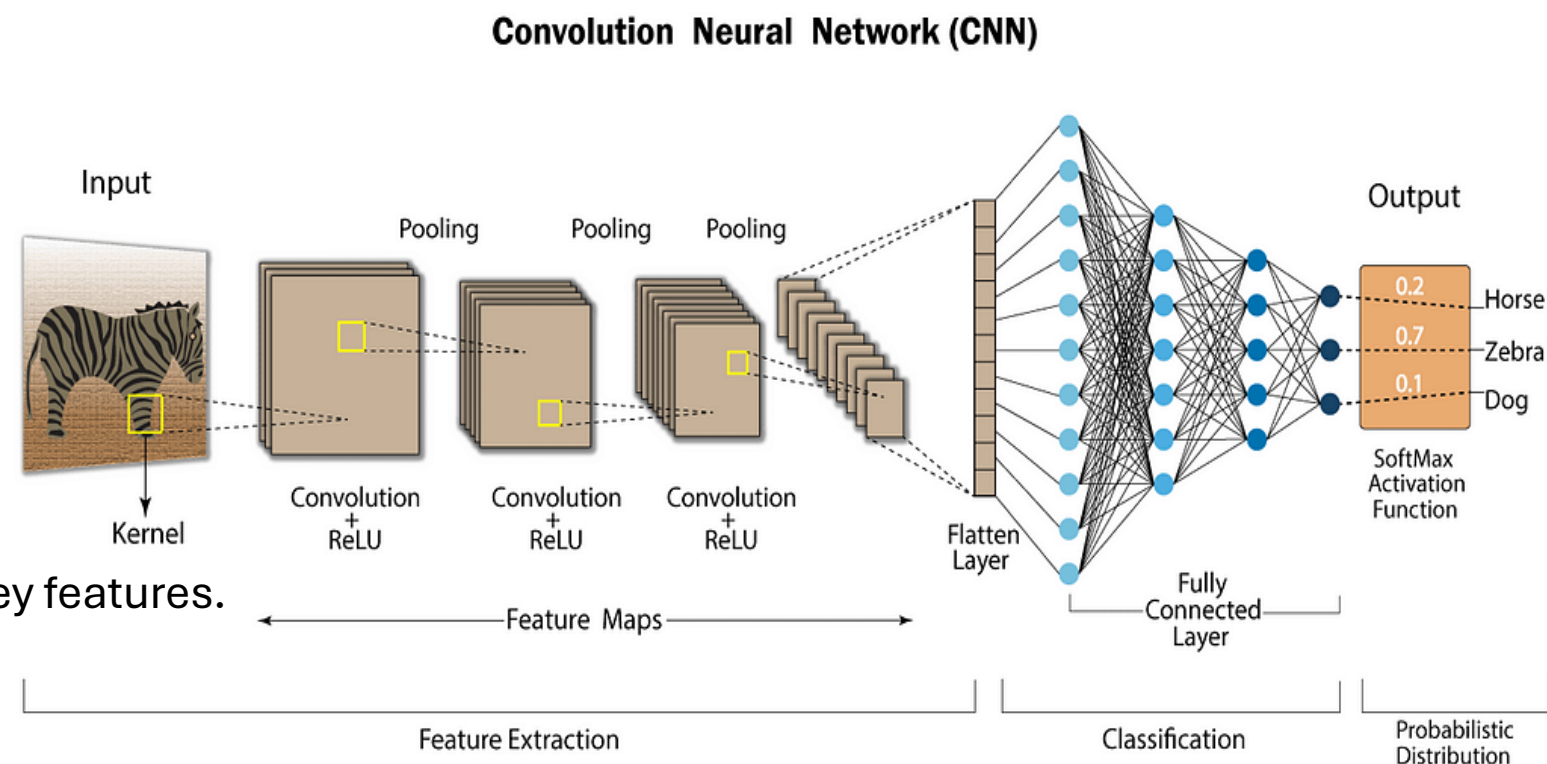
Introduces non-linearity (commonly ReLU).

- **Pooling Layer:**

Reduces dimensionality and emphasizes key features.

- **Fully Connected Layer:**

Performs the final classification.



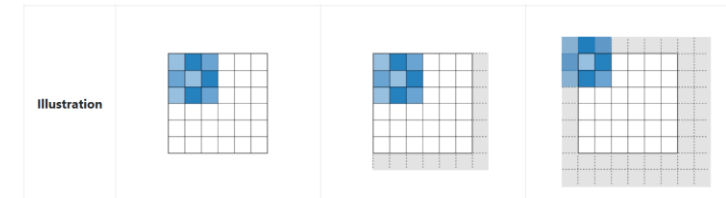
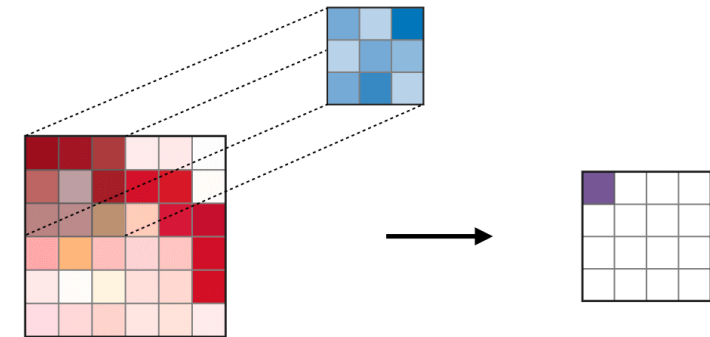
CNN: Convolutional Layer

This layer is responsible for extracting important features from the input image using a set of learnable filters/kernels (kernel are learned parameters!!)

- Applies a small learnable filter across the input image.
- Detects features like edges, corners, and textures.
- Uses the same weights at each location (weight sharing).

Its hyperparameters include:

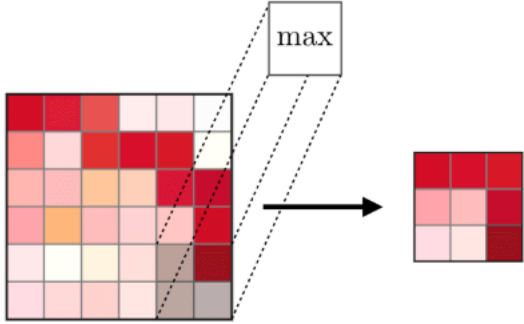
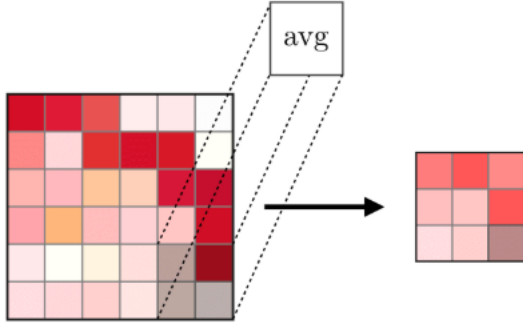
- Padding (P): adds a border of zeros around the input image. This helps control the spatial size of the output and preserves edge information when applying filters.
- Filter size (F): determines the spatial area each filter covers (e.g., 3×3)
- Stride (S): controls how far the filter moves across the image (e.g. 1 pixel) — larger strides reduce output size



Adding an activation function to the activation layers add nonlinearity to the network

CNN: Pooling

Pooling layers follow convolution layers and are used to reduce the spatial dimensions of feature maps while retaining important information.

Type	Max pooling	Average pooling
Purpose	Each pooling operation selects the maximum value of the current view	Each pooling operation averages the values of the current view
Illustration		
Comments	• Preserves detected features • Most commonly used	• Downsamples feature map • Used in LeNet

CNN

Benefits:

- Feature extraction: Automatically learns features from raw image data, reducing manual preprocessing.
- Spatial invariance: Can recognize patterns regardless of their location, orientation, or size.
- Robust to noise: Performs well even when input images are cluttered or noisy.
- Transfer learning: Pre-trained CNNs can be adapted to new tasks with less data and compute.
- High performance: Achieves state-of-the-art results in tasks like classification, detection, and segmentation.

Limitations:

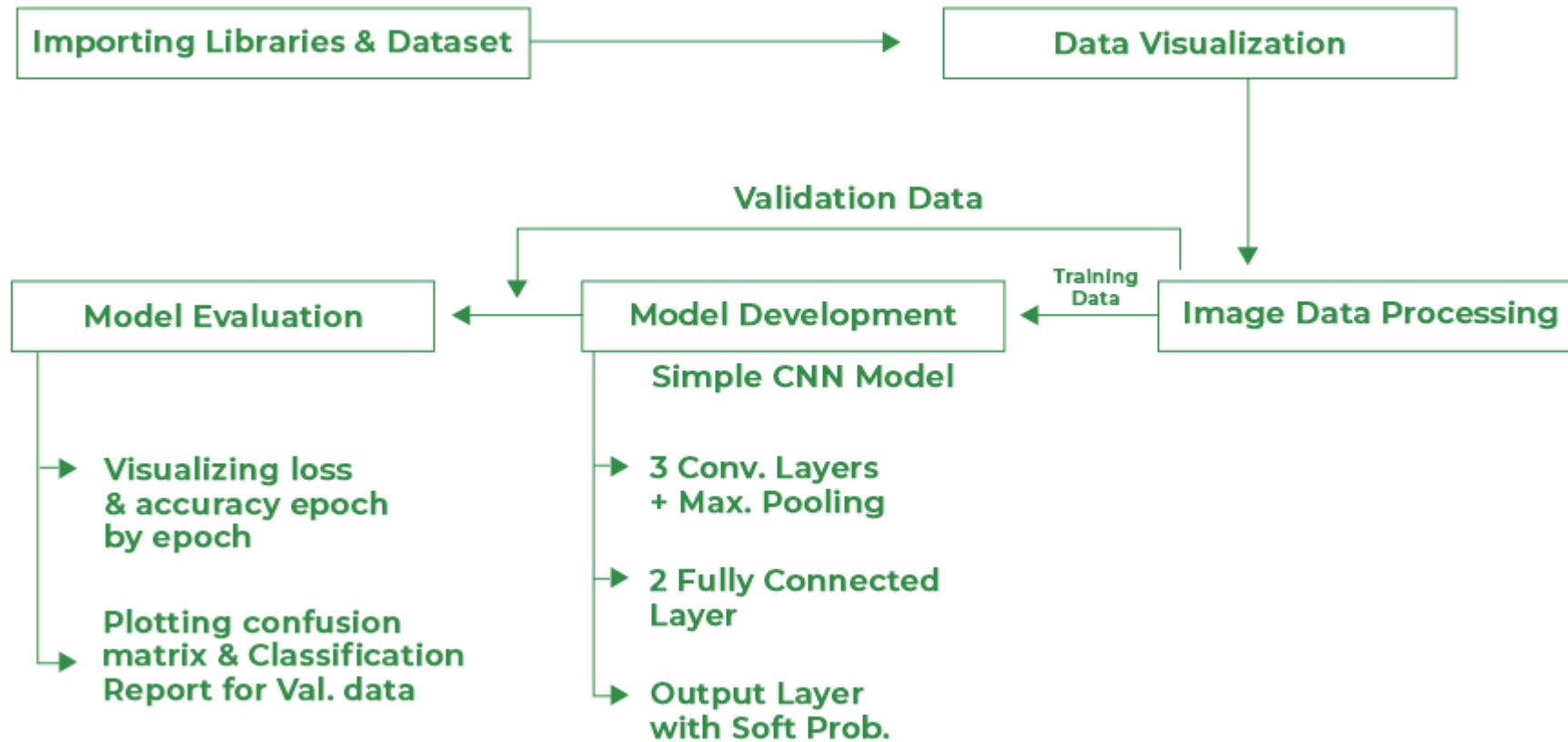
- Computational cost: Training deep CNNs can require a lot of resources and time.
- Overfitting: Prone to memorizing training data when datasets are small.
- Lack of interpretability: It's difficult to explain how or why a CNN made a specific decision.
- Structured input required: Works only with grid-like data (e.g., images), not irregular formats

Training a CNN

We will do the following steps in order:

- Load data (for pre defined dataset use torchvision)
- Clean your data: normalization, augmentation, resize, etc
- Extract features log-mel spectro
- Define a Convolutional Neural Network (number of con2D, pooling)
- Define a loss function to compare predictions to true labels
- Train the network on the training data
 - Compute the gradients of the loss with respect to the model parameters (weights and biases) using backpropagation.
 - Adjust the weights and biases using an optimizer (e.g., SGD, Adam) to minimize the loss.
 - Loop over all batches for multiple epochs until the model performance stabilizes.
- Test the network on the test data

Training a CNN



Overfitting

When a model learns the training data too well — including noise and details that don't generalize.

- Results in good training accuracy but poor performance on unseen (test) data.

How to prevent or reduce overfitting?

- More training data: Exposes the model to more variations.
- Data augmentation: Randomly change training audio (noise, effect, ...).
- Regularization: Add penalty terms to the loss (e.g., L2 weight decay).
- Dropout: Randomly disable some neurons during training.
- Early stopping: Stop training when validation loss starts to rise.
- Batch normalization: Helps by regularizing and stabilizing training.
- Simpler model: Use fewer layers or neurons if the model is too complex for the data.

Batch normalization

Applied after convolutional or fully connected layers:

Normalizes intermediate activations to zero mean and unit variance

Why?

- **Stabilizes training:** reduces fluctuations in layer outputs, helping the model learn more consistently.
- **Reduces internal covariate shift:** Keeps the input distribution of each layer more stable during training.
- **Allows higher learning rates:** Makes training more robust to large updates, speeding up convergence.
- **Acts as a form of regularization:** Adds slight noise from mini-batch stats, which helps prevent overfitting.

Recommended in most CNNs for audio/image tasks

Pre trained model

Classification Models:

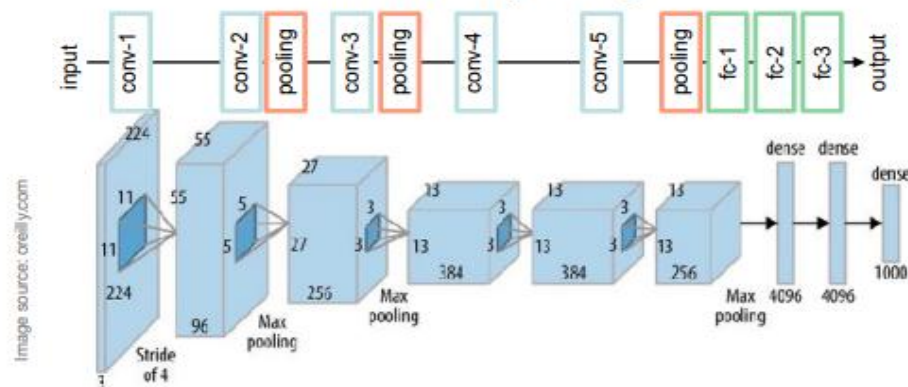
- VGGish (Google): Audio tagging
- YAMNet: EfficientNet-based, pretrained on AudioSet
- Wav2Vec2: Self-supervised model, usable for audio embeddings

SED-Specific Models:

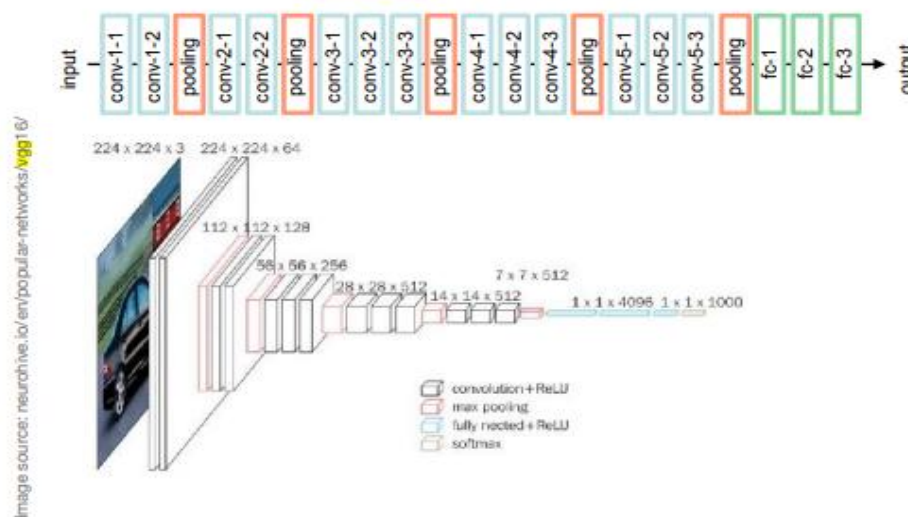
- PANNs: CNN-based, weakly-supervised
- CRNN (Convolutional Recurrent): Temporal modeling
- HTS-AT: Transformer-based audio tagging
- AST (Audio Spectrogram Transformer): Vision Transformer adapted to audio

Pre trained model

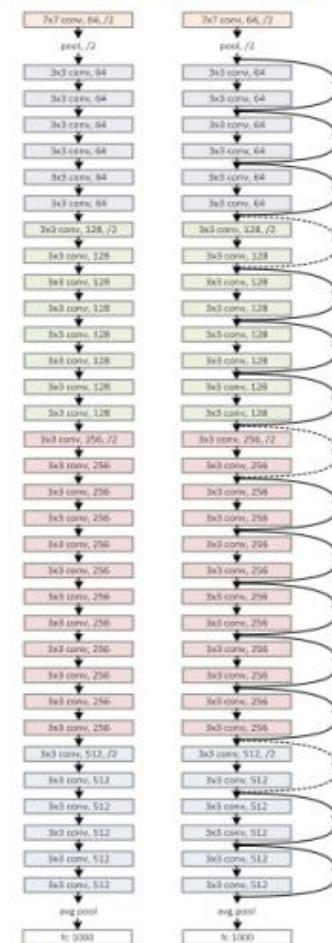
AlexNet (2012)



VGG-16 (2015)



ResNet (2016)



VGG-19

ResNet-34

PANNs

What are PANNs?

Pretrained CNN models for audio tagging & SED, trained on AudioSet (2M+ clips)

Model Types:

- CNN6, CNN10, CNN14 — progressively deeper CNNs
- Wavegram-CNN, Cnn14-ResNet, MobileNet-based variants available

Input Features:

- Log-Mel spectrograms (usually 64 Mel bins)

Outputs:

- Clip-level: Tagging (is "dog bark" present?)
- Frame-level: When did "dog bark" happen?

Create a Custom Dataset?

Why Do We Need `MyCustomDataset`? When working with audio datasets like UrbanSound8K:

Raw Data:

- - `.csv` file with labels and filenames
- - Folders of `.wav` audio clips

CNN need:

- Tensors as input, not file paths
- Uniform shape and format (e.g., [batch, channels, freq, time])

`MyCustomDataset` Solves These Challenges:

- Inherits from `torch.utils.data.Dataset`
- Dynamically loads each `.wav` file
- Applies **audio transformations** (e.g., log-Mel spectrograms)
- Pads/truncates all signals to same duration
- Returns `(spectrogram\_tensor, class\_label)` ready for training

Benefits:

- Memory-efficient (on-the-fly loading)
- Customizable (sampling rate, transforms)
- Integrates with `DataLoader` and GPU/CPU handling